28

BÖLÜM: ASENKRON İLETIŞIM, MESAJLAŞMA VE OLAY TABANLI MIMARI | MODULE: ASYNCHRONOUS MESSAGING & EVENT-DRIVEN ARCHITECTURE

Mesaj Teslimat Garantileri: At-Most-Once, At-Least-Once ve Exactly-Once (EOS)

Message Delivery Guarantees: At-Most-Once, At-Least-Once & EOS

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram

```
[ 1. AT-MOST-ONCE (En Fazla Bir Kez) ]
Mesajı al ──► Hemen ACK at ──► Mesajı İşle (Sunucu ÇÖKERSE -> MESAJ KAYBOLUR!)

[ 2. AT-LEAST-ONCE (En Az Bir Kez - Endüstri Standardı!) ]
Mesajı al ──► Mesajı İşle ──► DB'ye Yaz ──► ACK at (ACK ağda kaybolursa -> MÜKERRER İŞLENİR!)
(Çözüm: Tüketiciyi IDEMPOTENT tasarlamak!)

[ 3. EXACTLY-ONCE SEMANTICS (EOS - Tam Olarak Bir Kez) ]
İki Fazlı Onay (2PC) + Kafka Transaksiyonel Producer + Read-Committed İzolasyonu.
(En güvenli ancak en maliyetli ve karmaşık modeldir)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Birine mektup göndermenin 3 yolu: 1) At-Most-Once (Aptal Postacı): Mektubu posta kutusuna atar ve hemen arkasını dönüp gider. Mektup yağmurda çürürse çürür; asla tekrar denemez (Kayıp olabilir). 2) At-Least-Once (İnatçı Postacı): Kapıyı çalar, mektubu verir. Ancak siz 'Aldım' demeden önce gök gürültüsünden sesinizi duymazsa ertesi gün aynı mektuptan bir tane daha getirir! Mektup asla kaybolmaz ama iki kez teslim edilebilir (Mükerrerlik). 3) Exactly-Once (Noter Tasdikli Postacı): İki tarafın da imza attığı, noter kütüğüne işlenen kusursuz sistemdir; mektup ne kaybolur ne de iki kez gelir!

🇬🇧 ENGLISH (ELI5)

Three ways to deliver a package: 1) At-Most-Once: Drop the parcel on the porch and run. If a porch pirate steals it, tough luck; nobody redelivers (messages can be lost). 2) At-Least-Once: Ring the bell and hand over the parcel. If lightning strikes and the courier doesn't hear your 'Thank you' receipt, they return tomorrow with a duplicate parcel! Nothing is ever lost, but duplicates happen. 3) Exactly-Once: A bonded courier and biometric notary signing the delivery ledger synchronously: the package is delivered exactly one time with zero loss and zero duplicates!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) At-Most-Once: Tüketici mesajı okur okumaz offset'i commit eder, sonra işler. İşleme sırasında çökerse mesaj kaybolur (Sıfır veri kaybı toleransı olan sistemlerde kabul edilemez). 2) At-Least-Once: Tüketici önce mesajı işler, veritabanına kaydeder, en son offset commit eder. Commit sırasında ağ koparsa mesaj tekrar tüketilir (Mükerrerlik riski). Bu model endüstri standardıdır ve tüketici katmanında Idempotency (tekilleştirme) ile çözülür. 3) Exactly-Once Semantics (EOS): Kafka'da `enable.idempotence=true` (PID + Sequence Number) ve `TransactionalId` ile iki fazlı onay (2PC) kullanılarak sağlanır. Kafka'dan Kafka'ya (Stream-to-Stream) akışlarda kusursuzdur; ancak harici bir veritabanına yazarken harici sistemin de 2PC desteklemesi gerekir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) At-Most-Once: Consumers commit offset before processing. If processing crashes, the message is lost forever (unacceptable for financial systems). 2) At-Least-Once: Consumers process payload and persist to DB before committing offsets. If the network drops prior to offset commit, the broker redelivers on restart. This is the enterprise industry standard, pairing at-least-once delivery with idempotent downstream consumers. 3) Exactly-Once Semantics (EOS): Enforced in Kafka via idempotent producers (Producer ID + monotonic Sequence Numbers) and atomic transactions across multiple topic partitions using a Two-Phase Commit transaction coordinator. While native for Kafka-to-Kafka stream pipelines, sinking to third-party databases requires distributed transactions (XA) or idempotent UPSERTs.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Idempotent Tüketici Deseni (At-Least-Once Güvenliği)
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;

public class IdempotentConsumer {
    // İşlenmiş benzersiz mesaj ID'leri tablosu (Redis veya DB Unique Constraint)
    private final Set<String> processedMessageIds = ConcurrentHashMap.newKeySet();

    public record Message(String messageId, String payload) {}

    public synchronized boolean processMessage(Message msg) {
        // Eğer bu mesaj ID'si daha önce başarıyla işlendiyse: PAS GEÇ!
        if (!processedMessageIds.add(msg.messageId())) {
            IO.println("⚠ MÜKERRER MESAJ TESPİT EDİLDİ! Pas geçiliyor: " + msg.messageId());
            return false;
        }

        // İlk kez işleniyor:
        IO.println("✔ Mesaj başarıyla işlendi: " + msg.payload());
        return true;
    }
}

void main() {
    var consumer = new IdempotentConsumer();
    var msg = new IdempotentConsumer.Message("msg_uuid_9981", "Bakiye Düş: 50 TL");

    consumer.processMessage(msg); // 1. Teslimat -> İşlenir
    consumer.processMessage(msg); // 2. Ağ Yeniden Denemesi (Retry) -> Tekilleştirilir!
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Bizim sistemimiz uçtan uca Exactly-Once çalışıyor' iddiasında bulunurken dikkatli olun! Dış dünyadaki bir SMS veya e-posta servisine mesaj gönderirken Exactly-Once fiziksel olarak imkansızdır (SMS atıldıktan sonra elektrik kesilirse SMS iki kez gidebilir). Mülakatçıya gerçek dünyada çözümün 'At-Least-Once + Idempotency' olduğunu söyleyin.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Be wary of blindly claiming end-to-end Exactly-Once across heterogeneous systems. Sending an SMS or email cannot be rolled back via two-phase commit. Reassure the interviewer that real-world production relies on At-Least-Once paired with Idempotent consumers.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- At-Least-Once teslimat, mesaj kayıplarını sıfırlamak için en güvenilir temel modeldir.
- Mükerrer teslimat riskini bertaraf etmenin anahtarı tüketiciyi Idempotent yapmaktır.
- Kafka EOS, akış işleme (Kafka Streams) içinde tam olarak bir kez işlemeyi garanti eder.

🎯 Key Takeaways for the Interview

- At-Least-Once is the pragmatic distributed foundation to guarantee zero data loss.
- Pairing At-Least-Once with idempotent consumer logic neutralizes duplicate deliveries.
- Kafka transactional APIs guarantee Exactly-Once within internal stream processing topologies.